קטע קוד A

```
#include <iostream>
using namespace std;

#include <string>
#include <vector>

class Shape;

class Shapeable
{
public:
	virtual void toOs(ostream& os) const = 0;

	friend ostream& operator<<(ostream& os, const Shapeable& s) {
		s.toOs(os);
		return os;
	}
};

class Shape : public Shapeable
{
	int numOfPoints;
	string name;

public:
	Shape(int numOfPoints=3, const string& name="Triangle")
				: numOfPoints(numOfPoints), name(name) {}
	Shape(const Shape& other)
				: name(other.name), numOfPoints(other.numOfPoints)
	{
		cout << "Copying shape\n";
	}

	virtual ~Shape()
	{
		cout << "~Shape for " << name << endl;
	}

	virtual void toOs(ostream& os) const override
	{
		os << "The shape is " << name << ", numOfPoints=" << numOfPoints;
	}

	operator int() { return numOfPoints; }
};

class Circle : public Shape
{
	int radius;

public:
	Circle(int radius) : Shape(0, "Circle"), radius(radius) {}

	virtual void toOs(ostream& os) const override
	{
		Shape::toOs(os);
		os << ", radius=" << radius;
	}
};
```

```
class ShapeProperty : public Shapeable
{
	Shapeable* theShape;
public:
	ShapeProperty(Shapeable* theShape) : theShape(theShape) {}

	virtual void toOs(ostream& os) const override
	{
		os << *theShape;
	}

	Shapeable* getShapeable() { return theShape; }
};

class ShapeWithBorder : public ShapeProperty
{

	int borderLength;
public:
	ShapeWithBorder(Shapeable* theShape, int borderLength)
		: ShapeProperty(theShape), borderLength(borderLength) {}

	virtual void toOs(ostream& os) const override
	{
		ShapeProperty::toOs(os);
		os << ", border=" << borderLength;
	}

	void setBorderLength(int len) { borderLength = len; }
};

class ShapeWithColor : public ShapeProperty
{
	string color;
public:
	ShapeWithColor(Shapeable* theShape, const string& color)
		: ShapeProperty(theShape), color(color) {}

	virtual void toOs(ostream& os) const override
	{
		ShapeProperty::toOs(os);
		os << ", color=" << color.c_str();
	}

	void setColor(const string& color) { this->color = color; }
};
```

להלן מספר main אפשריים:

mainA1

```
void main()
{
	Shapeable* s2 = new ShapeWithBorder(
		new ShapeWithColor(
			new Circle(5), "red"), 2);
	cout << *s2 << endl;

	cout << endl;
}
```

mainA2

```
void main()
{
	vector<Shape> shapes;
	shapes.reserve(3);
	shapes.push_back(Shape());
	ShapeWithBorder sb(&shapes[0], 5);
	cout << sb << endl;
}
```

mainA3

```
void main()
{
	vector<Shape> shapes;
	shapes.reserve(5);
	shapes.push_back(Shape());
	shapes.push_back(Shape(4, "Rectangle"));
	shapes.push_back(Circle(5));

	vector<Shape>::iterator itr    = shapes.begin();
	vector<Shape>::iterator itrEnd = shapes.end();
	int sum = 0;
	for (; itr != itrEnd; ++itr)
		sum += *itr;

	cout << "------------\n";
	cout << sum << endl;
	cout << "------------\n";
}
```

mainA4

```
void main()
{
	Shapeable* s1 = new ShapeWithBorder(
		new ShapeWithColor(
			new Circle(5), "red"), 2);
	cout << "-----------\n";

	ShapeWithBorder* s2 = new ShapeWithBorder(*(ShapeWithBorder*)s1);
	cout << "-----------\n";
	ShapeWithColor* sc =
	   dynamic_cast<ShapeWithColor*>
		(dynamic_cast<ShapeProperty*>(s2)->getShapeable());

	if (sc != nullptr)
		sc->setColor("green");

	s2->setBorderLength(8);

	cout << "1 -----------\n";
	cout << *s1 << endl;
	cout << "2 ------------\n";
	cout << *s2 << endl;
	cout << "3 -----------\n";
	cout << *sc << endl;
	cout << "4 -----------\n";

	delete s1;
	delete s2;
}
```

קטע קוד B

```
#include <iostream>
using namespace std;

template<class T>
class MyClass
{
protected:
	T t1;
public:
	MyClass(const T& t) : t1(t)
	{
		cout << "Creating: " << *this << "\n";
	}

	virtual ~MyClass()
	{
		cout << "Destroying: " << *this << "\n";
	}

	virtual void foo() const
	{
		cout << "In MyClass::foo\n";
	}

	friend ostream& operator<<(ostream& os, const MyClass& c)
	{
		os << typeid(c).name() << " with " << typeid(c.t1).name()
			<< "=" << c.t1;
		return os;
	}

	const T& getT() const { return t1; }

	int operator+(const MyClass& other) const
	{
		return t1 + other.t1;
	}
};


template<class T>
class YourClass : public MyClass<T>
{
	T t2;
public:
	YourClass(const T& t) : MyClass(t)
	{
		cout << "Creating " << *this << "\n";
	}

	virtual ~YourClass()
	{
		cout << "Destroying: " << *this << "\n";
	}

	virtual void foo() { cout << "In YourClass::foo\n"; }

	T operator+(const YourClass& other) const { return t2 + other.t2; }
};
```

```
class One
{
private:
	static int theOne;
public:
	One()
	{
		++theOne;
	}

	int operator++()
	{
		++theOne;
		return theOne;
	}

	int operator++(int)
	{
		int temp = theOne;
		++theOne;
		return temp;
	}

	operator int() const { return theOne; }
};

int One::theOne = 0;

void main()
{
	One o1;
	MyClass<One> mc1(o1);
	cout << "-----------\n";

	YourClass<One> yc1(mc1.getT());
	cout << "-----------\n";

	MyClass<One>* yc2 = new YourClass<One>(yc1);
	cout << "-----------\n";
	/* put questions code here */

	cout << "-----------\n";

	delete yc2;
}
```

קטע קוד C

```
#include <iostream>
using namespace std;

template <class T>
class ClonePtr
{
private:
	T* ptr;
public:
	explicit ClonePtr(T* p = nullptr) : ptr(p) {}

	~ClonePtr() { delete ptr; }

	ClonePtr(const ClonePtr& p) : ptr(nullptr)
	{
		*this = other;
	}

	ClonePtr(ClonePtr&& other): ptr(other.ptr)
	{
		other.ptr = nullptr;
	}

	ClonePtr& operator=(const ClonePtr& other)
	{
		if (this != &other)
		{
			delete ptr;
			if (other.ptr != nullptr)
				ptr = other.ptr->clone();
		}
		return *this;
	}

	T& operator*() const { return *ptr; }
};
```

בהצלחה !